

Runtime Block Statistics RTL Module

Summary

This design implements a synthesizable Verilog module that processes unsigned 16-bit samples arriving at one sample per clock cycle. Samples are grouped into blocks whose length is provided using an 8-bit `block_length` input.

For each completed block, the module produces the minimum, maximum and mean. The mean is divided into an integer component and, as an extension, a value representing the fractional component to two decimal places.

The design assumes that a sample is available on every processing clock cycle. Results are available and valid for one clock cycle at the end of each block, during which no new sample is accepted.

Justifications

A sample width of 16 bits was used, with an 8-bit `block_length` allowing blocks of up to 255 samples. Consequently, 24 bits have been allocated to the internal `sum` value to avoid overflow. The maximum possible value is 16,711,425 which fits within 24 bits. Larger sample widths would increase the size of the internal datapath, while an 8-bit `block_length` already allows a relatively large range of block sizes.

Seven bits were used for the `remainder` output, as this can represent values from 0-127 and therefore comfortably fits the required 0-99 range for two decimal places. Scaling the remainder by 100 also allows `scaled_remainder` to remain below 16 bits.

12-bit sample values and a 4-bit `block_length` were considered to allow more of the internal values to remain within 16 bits. However, 16-bit samples and an 8-bit block length gave me more flexibility and made it easier to test a wider range of values.

The arithmetic mean is computed using `sum / sample_count` rather than `sum / block_length`. This allows `block_length` to be changed dynamically without altering the divisor used for samples that have already been accumulated. If `block_length` is decreased below the current sample count, the block terminates early, but the mean is still calculated using the actual number of samples received.

Division-by-zero conditions and reset behaviour were accounted for. `min_sample` is initialised to its maximum possible value so that it is overwritten immediately by the first sample.

Partway through development, I converted an `always @*` block into continuous combinational assignments. This does not really change the hardware produced, but I found it made the split between sequential and combinational logic much clearer.

Dynamic division was selected because the specification permits arbitrary block lengths instead of limiting values to powers of two. Although this approach can increase hardware resource consumption compared with constant or power-of-two division, it offers greater flexibility.

A running calculation was chosen over storing the entire block in memory. Retaining only the active sum, minimum, maximum and `sample_count` ensures that the required internal storage remains

independent of `block_length`.

The running minimum and maximum each require only a comparison and register update for every sample. Similarly, the running sum requires one addition per sample.

The average is calculated only after the final value has been received. This keeps the per-sample datapath relatively simple.

Verification

I verified the module using a Verilog testbench with Icarus Verilog 12.0 on EDA Playground. I initially tested standard values and a single maximum-value sample. Using the EPWave waveform viewer allowed me to identify an initial issue with the reset logic, which I then fixed in the testbench.

As development progressed alongside the testbench, I included a test for the fractional component and a test for altering the block length during operation. A test that increases the block length was not included because, without a separate sample-valid input, the design treats every active processing clock cycle as containing a valid sample. If no new value were provided, the existing input value would simply be sampled repeatedly until the new `block_length` was reached.

The following cases were tested:

- Standard input values: 10, 20, 30 and 41.
- A block length of one with the maximum 16-bit sample value, 65535.
- Extreme values including 0 and 65535, while also testing the fractional mean output.
- Decreasing `block_length` during a block to verify that the mean still uses the actual internal `sample_count`.

RTL Code

```
1 // Compiled with edaplayground.com
2 // Tools & Simulators = Icarus Verilog 12.0
3 // -Wall -g2012
4 // CHECK Open EPWave after run
5
6 module avg_calc(
7     input clk,
8     input rst,
9     input [15:0] sample, // 16 bits
10    input [7:0] block_length,
11    output [15:0] max,
12    output [15:0] min,
13    output [15:0] avg,
14    output [6:0] remainder
15);
16
17 // Assumptions:
18 // 1. block_length can be increased, decreasing it will terminate the program earlier
19 //    with the avg being calculated with internal length variable
20 // 2. A new sample is presented on every active processing clock cycle.
21 // 3. No sample is accepted during the one-cycle result/reset period.
22
23 wire block_done; // If/Else driver of combinational logic
24 wire [14:0] scaled_remainder;
25
26 reg [23:0] sum; // Max possible value
27 reg [15:0] min_sample;
```

```

27 reg [15:0] max_sample;
28 reg [7:0] sample_count; // Just in case there are errors with block length
29
30
31
32 always @(posedge clk or negedge rst) begin
33     if(!rst || block_length == 0 || sample_count >= block_length) begin
34         sum <= 0;
35         min_sample <= 65535; // 2^16 (65536) - 1
36         max_sample <= 0;
37         sample_count <= 0;
38
39     end else begin
40         sum <= sum + sample;
41         if (min_sample > sample) begin
42             min_sample <= sample;
43         end
44         if (max_sample < sample) begin
45             max_sample <= sample;
46         end
47         sample_count <= sample_count + 1;
48     end
49 end
50
51 // Combinational output logic using continuous assignments
52
53 assign block_done = (block_length != 0) && (sample_count >= block_length); // If/Else
54
55 assign avg = block_done ? sum / sample_count : 0; // Driven by reg sample_count to avoid
56 faulty block_length values corrupting avg
57 assign scaled_remainder = block_done ? (sum % sample_count) * 100 : 0; // Resets on else
58 assign max = block_done ? max_sample : 0;
59 assign min = block_done ? min_sample : 0;
60 assign remainder = block_done ? scaled_remainder / sample_count : 0;
61 endmodule

```

Testbench

```

1  'timescale 1ns/1ps
2
3  module testbench;
4
5      // Inputs to game_top
6      reg clk;
7      reg rst;
8      reg [15:0] sample;
9      reg [7:0] block_length;
10
11     // Outputs from game_top
12     wire [15:0] max;
13     wire [15:0] min;
14     wire [15:0] avg;
15     wire [6:0] remainder;
16
17
18     // Initialising
19     avg_calc shd ( // synthesizable hardware module
20         .clk(clk),

```

```

21     .rst(rst),
22     .sample(sample),
23     .block_length(block_length),
24     .max(max),
25     .min(min),
26     .avg(avg),
27     .remainder(remainder)
28 );
29
30
31 // 10 ns clock period
32 always #5 clk = ~clk;
33
34
35 initial begin
36
37     clk = 0;
38     sample = 0;
39     block_length = 4;
40
41     rst = 0; // Reset between tests because every active clock cycle is treated as a
42             valid sample, so excess gaps can be treated as valid results
43     @(negedge clk);
44     rst = 1;
45
46     // TEST 1 - STANDARD INPUT
47
48     sample = 10;
49
50     @(negedge clk);
51     sample = 20;
52
53     @(negedge clk);
54     sample = 30;
55
56     @(negedge clk);
57     sample = 41;
58
59     @(negedge clk);
60
61     $display("");
62     $display("TEST 1: STANDARD INPUT");
63     $display("MAX = %0d (expected 41)", max);
64     $display("MIN = %0d (expected 10)", min);
65     $display("AVG = %0d (expected 25)", avg);
66     $display("REMAINDER = %0d (expected 25)", remainder);
67
68     rst = 0;
69     @(negedge clk);
70     rst = 1;
71
72
73     // TEST 2 - BLOCK LENGTH OF 1 + MAX VALUE
74
75     block_length = 1;
76     sample = 65535;
77
78     @(negedge clk); // Only 1 cycle
79

```

```

80     $display("");
81     $display("TEST 2: BLOCK LENGTH OF 1 + MAX VALUE");
82     $display("MAX = %0d (expected 65535)", max);
83     $display("MIN = %0d (expected 65535)", min);
84     $display("AVG = %0d (expected 65535)", avg);
85     $display("REMAINDER = %0d (expected 0)", remainder);
86
87
88     rst = 0;
89     @(negedge clk);
90     rst = 1;
91
92
93     // TEST 3 - EXTREME VALUES + FRACTION
94
95
96     block_length = 3;
97
98     sample = 0;
99
100    @(negedge clk);
101    sample = 65535;
102
103    @(negedge clk);
104    sample = 1;
105
106    @(negedge clk);
107
108    $display("");
109    $display("TEST 3: EXTREME VALUES + FRACTION");
110    $display("MAX = %0d (expected 65535)", max);
111    $display("MIN = %0d (expected 0)", min);
112    $display("AVG = %0d (expected 21845)", avg);
113    $display("REMAINDER = %0d (expected 33)", remainder);
114
115    rst = 0;
116    @(negedge clk);
117    rst = 1;
118
119
120    // TEST 4 - BLOCK LENGTH DECREASED MID-BLOCK
121
122    block_length = 5;
123
124    sample = 10;
125
126    @(negedge clk);
127    sample = 20;
128
129    @(negedge clk);
130    sample = 30;
131
132    @(negedge clk);
133
134    // sum = 60, sample_count = 3
135    // Decreasing block length below sample_count terminates the block
136    block_length = 2;
137
138    #1; // Allow combinational outputs to update otherwise doesn't work
139

```

```

140      // This test is to show that the internal calculations are insulated from changes
141      // to block_length
142
143      $display("");
144      $display("TEST 4: BLOCK LENGTH DECREASED MID-BLOCK");
145      $display("MAX = %0d (expected 30)", max);
146      $display("MIN = %0d (expected 10)", min);
147      $display("AVG = %0d (expected 20)", avg);
148      $display("REMAINDER = %0d (expected 0)", remainder);
149
150      $display("");
151      $display("ALL TESTS COMPLETE");
152
153      #10;
154      $finish;
155
156  end
157
158
159  // Waveform output
160  initial begin
161      $dumpfile("dump.vcd");
162      $dumpvars(0, testbench);
163  end
164
165 endmodule

```